- BigDecimal:ROUND_FLOOR—Always round toward the lower number.
- BigDecimal::ROUND_HALF_DOWN—Round to the nearest value; the midpoint rounds down. So 8.5 rounds to 8, which is different than the normal default.
- BigDecimal::ROUND_HALF_EVEN—Round to the nearest value; the midpoint rounds to whichever neighbor is an even number. So 8.5 rounds to 8, but 9.5 rounds to 10. This is sometimes called "banker's rounding."
- BigDecimal::ROUND_HALF_UP—Round to the nearest value; the midpoint rounds up. This is the default you were probably taught in school.
- BigDecimal:ROUND_UP—Always round away from zero, so 8.2 rounds to 9.

Why Do Bankers Round Differently?



If you are adding up a very large set of numbers, one thing you want is for the sum to be essentially the same no matter where you round the value. Using traditional rounding, the fact that the midpoint rounds up will tend to bias rounded numbers to have a higher sum than the same set of numbers with the values just truncated, making the sum less accurate when rounded. Using banker's rounding, the midpoint number tends to go up or down more or less equally, so the rounded number does not have a biased sum relative to the truncated sum.

Random and SecureRandom

Random numbers are an important part of games and cryptographic security, and Ruby has a few different ways to get randomness. The easiest is the method Kernel#rand. Because it's available in Kernel, you can call rand anytime. If you call rand with no arguments, you get a pseudo-random float greater than or equal to zero and less than one. If you call rand with an argument and the argument is an integer one or greater, then you get a pseudo-random integer greater than or equal to zero and less than the argument. Note this means that rand(1) will always equal 0.

Non-integer arguments are converted using to_i.abs, meaning that negative arguments will return positive values, and floating-point arguments will be truncated. If arg.to_i.abs is equal to zero, then rand reverts to its no-argument behavior.

If the argument is a range, rand returns a value within that range. If both ends of the range are integers, the result will be an integer. If either end is a float, the result will also be a float.

Often you may want to have a repeatable sequence of random numbers. In testing or in debugging it can be useful to know that the random numbers are consistent from run to run. You can do this in Ruby with the Kernel method srand. If you call srand without an argument, it generates a seed based on the operating system's randomizer, but if you call it with an integer argument, it uses that number as the seed, and then it will provide a replicable stream of random numbers. Using the same seed later on will result in the same set of numbers.

Ruby also provides an object-oriented interface to the random number generator. The Random class provides basically the same features as rand, but allows you to have multiple streams. The formatter module, which gets mixed in with require random/formatter, gives you a set of methods on Random to produce structured random output.

You use Random by creating a new instance with Random.new, which takes an optional argument, which is the seed, exactly as described in srand. You can then get new random numbers with rand (and recover the seed with seed):